SQL Injection & HTTP Flood Attacks in IIoT

Problem, Impact, Detection Features (Edge-IIoTset) and Mitigation

	SQL Injection	HTTP Flood
Edge-IIoTset category	Injection attacks	DoS/DDoS attacks
OSI layer	Layer 7 (application / database)	Layer 7 (application)
Goal	Steal, modify or destroy data; bypass login	Exhaust server resources; deny service
Main impact	Confidentiality & integrity	Availability
Key fix	Parameterized queries + WAF + ML-IDS	Rate limiting + WAF/CDN + SDN blocking + ML-IDS

1. SQL Injection Payload Problem

What it is

IIoT systems usually expose a web dashboard or API (e.g. a ThingsBoard-style platform) backed by a database. If the application inserts user input directly into a SQL query, an attacker can supply input that the database executes as **SQL code** instead of treating it as data.

```
Normal request : /login?user=admin&pass=1234
Malicious input: user = admin' OR '1'='1
Resulting query: SELECT * FROM users WHERE user='admin' OR '1'='1'
                 -> condition is always TRUE, login is bypassed
```

Why it is a problem in IIoT

- **Data theft:** sensor readings, device credentials and configuration can be exposed.
- **Data tampering:** fake pH or water-level values lead to wrong decisions in the plant.
- **Takeover:** in the worst case the attacker can drop tables or control the platform.
- **Hard to see:** the attack hides inside normal-looking HTTP traffic on port 80/443, so basic firewalls allow it.

How it appears in Edge-IIoTset features

Feature	Indicator of SQL injection
http.request.uri.query	Unusual characters / keywords: ' -- OR UNION ;
http.request.full_uri / http.request.method	Abnormal URI structure or method usage
tcp.len / http.content_length	Request sizes that do not match normal traffic
tcp.payload	Carries the injected content

Mitigation

Layer	Technique
Application (prevention)	Parameterized queries / prepared statements (primary fix); input validation and whitelisting; ORM frameworks
Database	Least-privilege accounts (no admin rights for the web app); stored procedures; generic error messages that leak nothing
Network	Web Application Firewall (WAF) with SQLi rule sets, e.g. ModSecurity
Detection (my research)	ML-based IDS that learns normal HTTP request patterns and flags abnormal query structures, including new payloads that signatures miss

2. HTTP Flood Problem

What it is

An HTTP flood is a **Layer 7 DDoS attack**. A botnet of compromised devices sends very large numbers of valid-looking HTTP GET/POST requests to a server. Each request forces the server to do real work (load pages, query the database, render responses) until it can no longer serve legitimate users.

Why it is harder than other floods

- Unlike SYN or UDP floods, **each request looks legitimate**: the TCP handshake completes and the HTTP is valid.
- It does not need huge bandwidth; a moderate rate against heavy pages can crash the server.
- IoT devices are easily recruited into botnets (Mirai-style malware), giving attackers many sources.

Impact in IIoT

- The monitoring dashboard or cloud/fog server becomes unresponsive.
- Sensor data cannot be uploaded, creating gaps in real-time monitoring.
- Alarms (e.g. from the flame sensor) may arrive late, creating a **safety risk**.
- Edge and fog nodes have limited resources, so they are overwhelmed even faster.

How it appears in Edge-IIoTset features

Feature	Indicator of HTTP flood
ip.src_host	Sudden rise in request rate from one or many sources
http.request.method / http.request.full_uri	Repeated GET/POST requests to the same URI
tcp.connection.syn / tcp.connection.synack	Many connection setups in a short time window
frame.time / tcp.len	Very short inter-arrival times and similar packet sizes

Mitigation

Layer	Technique
Rate control	Rate limiting per IP or session; request throttling; connection limits
Filtering	WAF rules; IP reputation and blacklists; geo-blocking; CAPTCHA or JavaScript challenges to separate bots from humans
Infrastructure	Load balancers, CDNs and reverse proxies (e.g. Nginx) to absorb traffic; auto-scaling in the cloud layer
SDN (Edge-IIoTset testbed)	The ONOS SDN controller drops or redirects attacking flows at the switch once they are detected
Detection (my research)	ML/DL IDS at the edge or fog spots the flood early from traffic features, then triggers blocking

3. Link to My Research

> *"Traditional defences such as WAF rules and rate limits are signature- or threshold-based. They miss new SQL injection variants and low-rate HTTP floods that resemble normal traffic. My approach uses a federated learning-based intrusion detection model trained at the edge on device traffic. It learns normal HTTP behaviour and detects these attacks without sending raw data to a central server, preserving privacy and reducing latency."*

Step	Action
1. Detect	Federated IDS on edge/fog nodes classifies traffic as normal or attack
2. Alert	Detection result and attack type are sent to the monitoring platform
3. Block	SDN controller / firewall / WAF drops the malicious flows or source IPs
4. Learn	Model is updated in the next federated round without sharing raw data